

Assignment 3

1. include Irvine32.inc
.data

dividend word 0D4A4h
divisor word 0Ah

.code

main proc

mov ax, dividend

mov bx, divisor

call recursiveDivide

movzx eax, ax

call writeHex

call crlf

exit

main endp

recursiveDivide proc

cmp ax, 05h

jbe stop

push ax

push bx

xor dx, dx

; clear dx

div bx

call recursiveDivide

pop bx

pop ax

stop:

ret

```
recursiveDivide endp  
end main.
```

```
2. include Irvine32.inc  
    .data
```

```
prompt    byte "Enter a string: ", 0  
input     byte 200 dup(?)  
countA    dword 0  
countE    dword 0  
countI    dword 0  
countO    dword 0  
countU    dword 0
```

```
    .code
```

```
main proc
```

```
    mov edx, offset prompt  
    call writeString  
    mov edx, offset input  
    mov ecx, sizeof input  
    call readString
```

```
    mov esi, offset input
```

```
count:
```

```
    mov al, [esi]  
    cmp al, 0  
    je print
```


; converting upper case to lower case:

cmp al, 'A'

jb skip

cmp al, 'Z'

ja lower

add al, 32

lower:

cmp al, 'a'

jb skip

cmp al, 'z'

ja skip

cmp al, 'a'

inc countA

jmp skip

cmp al, 'e'

inc countE

jmp skip

cmp al, 'i'

inc countI

jmp skip

cmp al, 'o'

inc countO

jmp skip

cmp al, 'u'

inc countU

skip:

inc esi

jmp count

print:

mov eax, countA

call writeDec

call crlf

mov eax, countE

call writeDec

call crlf

mov eax, countI

call writeDec

call crlf

mov eax, countO

call writeDec

call crlf

mov eax, countU

call writeDec

call crlf

exit

main endp

end main

3. include Irvine32.inc

• data

string byte "###FAST", 0

char byte '#'

• code

main proc

mov esi, offset string


```
mov al, char  
call trimString
```

```
mov edx, offset string  
call writeString.  
call crlf  
exit
```

```
main endp
```

```
trimString proc
```

```
push edi
```

```
push ecx
```

```
mov edi, esi
```

```
mov ecx, 0
```

```
skip:
```

```
mov bl, [esi]
```

```
cmp bl, 0
```

```
je done
```

```
cmp bl, al
```

```
jne copy
```

```
inc esi
```

```
inc ecx
```

```
jmp skip
```

```
copy:
```

```
cmp ecx, 0
```

```
je done
```

```
mov edi, esi
```

```
sub edi, ecx
```

shift:

```
mov bl, [esi]
mov [edi], bl
inc esi
inc edi
cmp bl, 0
jne shift
```

done:

```
pop ecx
pop edi
ret
```

```
timString endp
end main
```

4. include irvine32.inc

.data

arr1 dword 5,5,5,5,5,9,8,7

arr2 dword 1,5,5,5,5,4,5,5

arr3 dword 5,1,5,5,5,5,5

arr4 dword 1,2,3,4,5,6,7,8

msg byte "Result = ", 0

.code

main proc

; test 1:

mov edx, offset msg

invoke FindFive, addr arr1, lengthof arr1


```

    call writeDec
    call crlf
    ; test 2:
    mov edx, offset msg
    call writeString
    invoke FindFive, addr an2, lengthof an2
    call writeDec
    call crlf
    ; test 3
    mov edx, offset msg
    call writeString
    invoke FindFive, addr an3, lengthof an3
    call writeDec
    call crlf
    ; test 4
    mov edx, offset msg
    call writeString
    invoke FindFive, addr an4, lengthof an4
    call writeDec
    call crlf
    exit

```

main endp.

FindFive proc uses ebx, ecx, edx esi

pArray: PTR dword

Size: dword

mov esi, ~~0~~ pArray

lengthof ans2
mov ecx, size
mov ebx, 0

check:

cmp ecx, 0
je NotFound

lengthof ans3
mov eax, [esi]

cmp eax, 5

jne reset

inc ebx

lengthof ans4
cmp ebx, 5

je found

jmp continue

reset:

mov ebx, 0

continue:

add esi, 4

dec ecx

jmp check

found:

mov eax, 1

ret

NotFound:

mov eax, 0

ret

FindFive endp

End main